RA26-Challenge 5: Solución Entrenamiento

Entity Manager, sistemas, sprites y escenas

ⓘ Cómo usar esta solución
Intenta resolver el ejercicio antes de mirar. Compara con tu solución: no tiene que ser idéntica. Debe cumplir el contrato y debes entender cada bloque.
Los comentarios con `\ | /` agrupan las instrucciones en bloques lógicos.

Ejercicio 1: man_entity_alloc: reservar una entidad (corregir el código)

🐞 Los errores

Tipo	Dónde	Corrección
E	`ld b, ENTITY_ARRAY_SIZE`	B es cuántas entidades quedan por mirar, no cuántos bytes tiene el array: `ld b, MAX_ENTITIES`.
E	`jr nz, .found`	Hay que ir a `.found` cuando la entidad NO está viva: `jr z, .found`.
N	`add hl, SIZEOF_ENTITY`	No existe `add hl, n`. El array está alineado: `add SIZEOF_ENTITY` sobre `A` (que ya es `L`).
F	falta `ld c, a` al empezar	La rutina usa `A` para calcular `L`: los componentes se pierden. Hay que guardarlos en `C` antes (`.found` los lee de `C`).
F	`.found`: falta `or CMP_ALIVE`	Hay que marcar la entidad como viva: `or CMP_ALIVE` antes de `ld [hl], a`.
E	`.found`: `or 1`	El éxito se devuelve con Z. `or 1` devuelve NZ (error). Hace falta algo que active Z: `xor a` o `cp a` (el contrato permite modificar `A`).

man_entity_alloc (corregida)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Looks for a free entity and reserves it.
;; 📥 INPUT:   A: components the new entity will have (CMP_SPRITE, ...)
;; 🔙 RETURNS: Z,  HL = address of the entity, and [HL] = A | CMP_ALIVE
;;             NZ, there is no free entity
;; ❌ May modify A, BC, HL. Must NOT modify DE
;; ⚠️ It does not change the other fields: the caller fills them in
man_entity_alloc::
    ld   c, a                   ;; \ C = components of the new entity
    ld   hl, entity_array       ;; | HL = first entity of the array
    ld   b, MAX_ENTITIES        ;; / B = entities left to check

.next
    bit  CMP_ALIVE_BIT, [hl]    ;; \ Is this entity alive?
    jr   z, .found              ;; / No: it is free. Take it

    ld   a, l                   ;; \ HL += SIZEOF_ENTITY
    add  SIZEOF_ENTITY          ;; | (the array is aligned:
    ld   l, a                   ;; / only L changes)

    dec  b                      ;; \ B--: keep looking
    jr   nz, .next              ;; / while there are entities left

    or   1                      ;; \ The array is full:
    ret                         ;; / A = 1, so NZ (no entity)

.found
    ld   a, c                   ;; \ A = the components of the new entity
    or   CMP_ALIVE              ;; | + the ALIVE bit
    ld   [hl], a                ;; / entity.CMP = A: it is reserved

    xor  a                      ;; \ Z: success
    ret                         ;; / (HL = the entity)
```

ⓘ Fíjate en

- El array está alineado a 256 bytes y mide 64: pasar de entidad en entidad solo cambia `L`. No uses `add hl, de`: el contrato prohíbe modificar `DE`.
- `bit CMP_ALIVE_BIT, [hl]` no modifica registros.

⚠ Errores típicos

Nos pasa a todos:

- Devolver el resultado al revés (`Z` con el array lleno). Quien llama espera `Z` = éxito.
- Olvidar `or CMP_ALIVE`: la entidad seguiría libre y `alloc` la entregaría otra vez.

Ejercicio 2: man_entity_for_each: llamar a una rutina por cada entidad

man_entity_for_each

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Calls a routine once for every entity that is alive.
;; 📥 INPUT: HL: address of the routine to call
;;           The routine receives DE = the entity, and it may modify
;;           every register: for_each looks after its own state.
;; ❌ May modify A, BC, DE, HL
;; ⚠️ Use "call call_hl" to call the routine whose address is in HL
man_entity_for_each::
    ld   de, entity_array       ;; \ DE = first entity
    ld   b, MAX_ENTITIES        ;; / B = entities left

.next
    ld   a, [de]                ;; \ A = entity.CMP
    bit  CMP_ALIVE_BIT, a       ;; | Is it alive?
    jr   z, .skip               ;; / No (it is free): skip it

    push bc                     ;; \ Save what the routine can change:
    push de                     ;; | B = counter, DE = entity,
    push hl                     ;; / HL = the routine

    call call_hl                ;; routine(DE = entity)

    pop  hl                     ;; \ Restore them
    pop  de                     ;; |
    pop  bc                     ;; /

.skip
    ld   a, e                   ;; \ DE += SIZEOF_ENTITY
    add  SIZEOF_ENTITY          ;; | (the array is aligned:
    ld   e, a                   ;; / only E changes)

    dec  b                      ;; \ B--: next entity,
    jr   nz, .next              ;; / while there are entities left

    ret
```

ⓘ Fíjate en

- Los tres `push` guardan lo que la rutina puede cambiar: contador, entidad y la propia rutina.
- `call call_hl` es un `call` normal a un `jp hl`: el `ret` de la rutina vuelve a la instrucción siguiente.

⚠ Errores típicos

Nos pasa a todos:

- Olvidar guardar `HL`: en la segunda vuelta llamas a una dirección cualquiera.
- Hacer los `pop` en el mismo orden que los `push`: cada registro recibe el valor de otro.

Ejercicio 3: sys_physics_one / sys_physics_update: mover y soltar lo que sale (corregir el código)

Los errores

Tipo	Dónde	Corrección
E	`ret nz` (primer filtro)	Solo se mueven las entidades CON PHYSICS: si el AND da 0, hay que salir: `ret z`.
N	`ld [hl+], c`	Solo `A` puede escribir en `[hl+]`: `ld [hl+], a`.
E	`sub b`	La nueva X es X + VX: `add b`.
F	falta `ld b, a` tras escribir la X	La nueva X debe quedar en `B` (la usan `sys_physics_limit_x` y la comprobación de bordes).
E	segundo `cp ENTITY_OUT_Y`	La X se compara con su límite: `cp ENTITY_OUT_X`.
E	`ret nc` tras el `cp` de la X	Se queda (y sale de la rutina) si X < ENTITY_OUT_X, que deja el carry a 1: `ret c`.

sys_physics_one (corregida)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves ONE entity: Y += VY, X += VX. Then, what happens at the edges depends on it:
;;  - The entities with INPUT (the ship) are controlled by the player: they
;;    cannot leave. Their limits are done by sys_physics_limit_x (given): jump to it.
;;  - The others are freed if they leave the screen (Y >= ENTITY_OUT_Y or
;;    X >= ENTITY_OUT_X). Note: a coordinate that goes below 0 wraps to 255.
;; Only works on entities with the PHYSICS component.
;; (A semi-generic system: ideally every entity would have its own limits
;;  and its own policy, but that is too much for this level.)
;; 📥 INPUT: DE: entity
;; ❌ May modify A, BC, DE, HL
sys_physics_one::
    ld   a, [de]                ;; \ A = entity.CMP
    and  CMP_PHYSICS            ;; | Does it have PHYSICS?
    ret  z                      ;; / No: not for this system
.get_entity_physics_data
    ld   hl, E_VY               ;; \ HL = &entity.VY
    add  hl, de                 ;; |
    ld   a, [hl+]               ;; | C = entity.VY (HL -> VX)
    ld   c, a                   ;; |
    ld   b, [hl]                ;; / B = entity.VX

    ld   hl, E_Y                ;; \ HL = &entity.Y
    add  hl, de                 ;; /
.update_physics
    ld   a, [hl]                ;; \ A = entity.Y
    add  c                      ;; | Y += VY
    ld   [hl+], a               ;; | (HL -> X)
    ld   c, a                   ;; / C = the new Y

    ld   a, [hl]                ;; \ A = entity.X
    add  b                      ;; | X += VX
    ld   [hl], a                ;; | (HL -> X, still)
    ld   b, a                   ;; / B = the new X

    ld   a, [de]                ;; \ A = entity.CMP
    and  CMP_INPUT              ;; | Does it have INPUT?
    jp   nz, sys_physics_limit_x ;; / Yes: it is controlled by the player: its limits
(given)
.check_boundaries_generic
    ld   a, c                   ;; \ A = the new Y
    cp   ENTITY_OUT_Y           ;; | Is Y >= ENTITY_OUT_Y?
    jr   nc, .leaves            ;; / Yes: it left through the bottom

    ld   a, b                   ;; \ A = the new X
    cp   ENTITY_OUT_X           ;; | Is X >= ENTITY_OUT_X? (below 0 wraps to 255)
    ret  c                      ;; / No: it stays (carry: X < ENTITY_OUT_X)

.leaves
    jp   man_entity_free        ;; It left the screen: it no longer exists
```

sys_physics_update (corregida)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves ALL the entities that have PHYSICS.
;; ❌ May modify A, BC, DE, HL
sys_physics_update::
    ld   hl, sys_physics_one
    jp   man_entity_for_each
```

Fíjate en

- `C` guarda la nueva `Y` y `B` la nueva `X`; `HL` queda en `X`: justo lo que espera `sys_physics_limit_x`.
- `cp ENTITY_OUT_X` es sin signo: una `X` que bajó de 0 vale 255 y queda fuera sin mirar el signo.

Errores típicos

Nos pasa a todos:

- Tratar la velocidad como positiva. Con `VX = -1` (`$FF`) la suma funciona sola.
- Liberar la nave al salir: hay que saltar a `sys_physics_limit_x` antes.

Ejercicio 4: sys_shot_spawn: crear el disparo (solo puede haber uno)

sys_shot_spawn

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Creates THE shot at a position. There can only be ONE shot: if it already
;; exists, or there is no free entity, it does nothing.
;; The entity is created from ShotTemplate data, and then Y and X are changed.
;; The system stores the address of the entity in shot_entity
;; Where the shot appears (D, E) = (Y, X) is a rule of the game: shot_spawn decides it.
;; 📥 INPUT:   D = Y, E = X
;; 🔙 RETURNS: Z, the shot has been created
;;             NZ, it has not (there was one, or there is no free entity)
;; ❌ May modify A, BC, HL. Does NOT modify DE
;; ⚠️ Use man_entity_create
sys_shot_spawn::
    ld   a, [shot_entity + 1]   ;; \ A = high byte of shot_entity
    or   a                      ;; | 0 means there is no shot
    ret  nz                     ;; / There is one already: NZ

    ld   hl, ShotTemplate       ;; \ Create it from its template
    call man_entity_create      ;; | (SPRITE + PHYSICS + COLLIDER, KIND_SHOT)
    ret  nz                     ;; / No free entity: NZ

    ld   a, l                   ;; \ shot_entity = HL:
    ld   [shot_entity], a       ;; | remember where the shot is
    ld   a, h                   ;; |
    ld   [shot_entity + 1], a   ;; /

    inc  hl                     ;; HL -> shot.Y

    ld   a, d                   ;; \ shot.Y = D
    ld   [hl+], a               ;; / (HL -> X)

    ld   a, e                   ;; \ shot.X = E
    ld   [hl], a                ;; /

    xor  a                      ;; \ Z: the shot has been created
    ret                         ;; /
```

ⓘ Fíjate en

- El sistema es el único que toca `shot_entity`: el juego decide DÓNDE aparece el disparo y el sistema lo crea.
- `man_entity_create` conserva `DE`.

⚠ Errores típicos

Nos pasa a todos:

- Guardar `H` antes que `L`: el puntero queda al revés.
- No devolver `Z` al crearlo: quien llama no sabe que hay disparo.

Ejercicio 5: sys_shot_update: comprobar que el disparo sigue ahí

sys_shot_update

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks that THE shot is still there, and if not, forgets it (shot_entity = 0).
;; sys_physics moves the shot and frees it when it leaves the screen, like any
;; other entity: this system does not move anything. The shot is still there only if
;; its entity is alive AND it is still a shot (KIND_SHOT): its slot may have been
;; freed, or taken by another entity (an asteroid, for example).
;; If there is no shot, it does nothing.
;; ❌ May modify A, HL
sys_shot_update::
    ld   a, [shot_entity]       ;; \ HL = *(shot_entity)
    ld   l, a                   ;; |
    ld   a, [shot_entity + 1]   ;; |
    ld   h, a                   ;; / = the shot

    or   a                      ;; \ Is there a shot?
    ret  z                      ;; / No: nothing to do

    bit  CMP_ALIVE_BIT, [hl]    ;; \ Is its entity alive?
    jr   z, .forget             ;; / No: it has been freed

    ld   a, l                   ;; \ HL -> shot.KIND
    add  E_KIND                 ;; | (the array is aligned:
    ld   l, a                   ;; / only L changes)

    ld   a, [hl]                ;; \ Is it still a shot?
    cp   KIND_SHOT              ;; | (another entity may have taken its slot)
    ret  z                      ;; / Yes: it is still there

.forget
    xor  a                      ;; \ shot_entity = 0:
    ld   [shot_entity], a       ;; | there is no shot
    ld   [shot_entity + 1], a   ;; / any more

    ret
```

ⓘ Fíjate en

- Hacen falta las dos condiciones: un hueco libre conserva su `KIND`, y un hueco ocupado puede ser un asteroide.
- `sys_shot_destroy` ya está hecha: la usa `game_enemy_hit` para quitar el disparo cuando acierta.

⚠ Errores típicos

Nos pasa a todos:

- Mirar solo `CMP_ALIVE`: un asteroide en el hueco del disparo pasaría por "el disparo".
- Mirar solo `KIND_SHOT`: una entidad libre conserva su `KIND` y pasaría por viva.

Ejercicio 6: sys_prerender_one: preparar el sprite de una entidad

sys_prerender_one

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Prepares ONE entity to be drawn: copies its sprite (Y, X, TILE, ATTR:
;; 4 bytes, in the same order as the OAM) to the shadow OAM, to the place
;; that shadow_cursor says, and moves the cursor 4 bytes on.
;; If the entity does not have CMP_SPRITE, it does nothing.
;; 📥 INPUT: DE: entity
;; ❌ May modify A, B, DE, HL
;; ⚠️ Use memcpy8
sys_prerender_one::
    ld   a, [de]                ;; \ Check if entity is valid for this system
    and  CMP_SPRITE             ;; | Without CMP_SPRITE: not for this system
    ret  z                      ;; /

    ld   hl, E_SPRITE           ;; \ HL = &entity.SPRITE
    add  hl, de                 ;; / = source: Y, X, TILE, ATTR

    ld   a, [shadow_cursor]     ;; \ DE = *(shadow_cursor)
    ld   e, a                   ;; | DE = destination: the cursor
    ld   a, [shadow_cursor + 1] ;; | Sprite should be copied there in shadow_oam
    ld   d, a                   ;; /

    ld   b, 4                   ;; \ Perform the copy of the sprite
    call memcpy8                ;; / Note: DE ends up 4 bytes further on

    ld   a, e                   ;; \
    ld   [shadow_cursor], a     ;; | *(shadow_cursor) = DE
    ld   a, d                   ;; | Address in shadow_ram where
    ld   [shadow_cursor + 1], a ;; / the next sprite should be written
    ret
```

ⓘ Fíjate en

- `HL` se calcula antes de cargar el cursor en `DE`, porque `DE` era la entidad.
- `memcpy8` deja `DE` 4 bytes más allá: es el nuevo cursor.

⚠ Errores típicos

Nos pasa a todos:

- Copiar la entidad entera (8 bytes): el sprite empieza en `E_SPRITE`, no en `CMP`.
- Olvidar guardar el cursor: todos los sprites se escriben en el mismo sitio.

Ejercicio 7: sys_prerender_update: preparar toda la OAM sombra

sys_prerender_update

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Prepares the whole shadow OAM (in RAM, so no hurry and no need for VBLANK):
;;  1. Fills it with zeros (Y = 0: every sprite hidden), so that
;;     the entities that no longer exist do not leave sprites behind.
;;  2. Puts shadow_cursor at the start of the shadow OAM.
;;  3. Prepares every entity with sys_prerender_one.
;; ❌ May modify A, BC, DE, HL
;; ⚠️ Use memset8 (clear) and man_entity_for_each
sys_prerender_update::
    ld   hl, shadow_oam         ;; \ HL = the shadow OAM
    xor  a                      ;; | A = 0 (Y = 0: sprite hidden)
    ld   b, SHADOW_OAM_SIZE     ;; | B = its size
    call memset8                ;; / Hide all the sprites

    ld   a, LOW(shadow_oam)     ;; \ shadow_cursor = shadow_oam:
    ld   [shadow_cursor], a     ;; |
    ld   a, HIGH(shadow_oam)    ;; |
    ld   [shadow_cursor + 1], a ;; / the first sprite goes first

    ld   hl, sys_prerender_one  ;; \ For every entity:
    jp   man_entity_for_each    ;; / sys_prerender_one(entity)
```

ⓘ Fíjate en

- Vaciar con ceros deja `Y = 0` en los sprites sobrantes: fuera de la pantalla.

⚠ Errores típicos

Nos pasa a todos:

- No vaciar: el sprite de una entidad liberada se queda en pantalla.

Ejercicio 8: sys_render_update: copiar la OAM sombra a la OAM

sys_render_update

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Shows the sprites: waits for VBLANK and copies the first SHADOW_OAM_SIZE bytes of
;; the shadow OAM (the sprites of the entities) to the OAM, which is at OAM_START
($FE00).
;; The OAM can only be written during VBLANK (and with the screen off).
;; This routine knows nothing about entities: it is a plain copy.
;; ❌ May modify A, B, DE, HL
;; ⚠️ Use wait_vblank and memcpy8
sys_render_update::
    call wait_vblank            ;; Wait for VBLANK: the OAM can be written from now on

    ld   hl, shadow_oam         ;; \ HL = the shadow OAM
    ld   de, OAM_START          ;; | DE = the OAM
    ld   b, SHADOW_OAM_SIZE     ;; | B = the size of the sprites of the entities
    jp   memcpy8                ;; / Copy them to the OAM
```

ⓘ Fíjate en

- Dos llamadas y una copia: la lógica de qué se dibuja está en el pre-render.

⚠ Errores típicos

Nos pasa a todos:

- No esperar al VBLANK: en una consola real los sprites se corrompen.

Ejercicio 9: sys_collision_overlaps: ¿se solapan dos entidades?

sys_collision_overlaps

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Do two entities overlap? They do if, in BOTH axes, the distance
;; between their positions is less than HIT_SIZE: |Y1 - Y2| < HIT_SIZE
;; and |X1 - X2| < HIT_SIZE.
;; 📥 INPUT:   DE: entity 1,  HL: entity 2
;; 🔙 RETURNS (in the carry, as in Challenge 4):
;;    C,  they overlap
;;    NC, they do not
;; ❌ May modify A, DE, HL
sys_collision_overlaps::
    inc  de                     ;; \ DE -> Y1
    inc  hl                     ;; / HL -> Y2

    ld   a, [de]                ;; \ A = Y1 - Y2
    sub  [hl]                   ;; /

    jr   nc, .y_positive        ;; \ Positive: it is already |dY|
    cpl                         ;; | Negative: |dY| = -dY
    inc  a                      ;; / (cpl + inc: two's complement)

.y_positive
    cp   HIT_SIZE               ;; \ Carry if |dY| < HIT_SIZE
    ret  nc                     ;; / Too far in Y: NC, no overlap

    inc  de                     ;; \ DE -> X1
    inc  hl                     ;; / HL -> X2

    ld   a, [de]                ;; \ A = X1 - X2
    sub  [hl]                   ;; /

    jr   nc, .x_positive        ;; \ Positive: it is already |dX|
    cpl                         ;; | Negative: |dX| = -dX
    inc  a                      ;; / (cpl + inc: two's complement)

.x_positive
    cp   HIT_SIZE               ;; \ Carry if |dX| < HIT_SIZE:
    ret                         ;; / C = they overlap, NC = they do not
```

ⓘ Fíjate en

- Los dos ejes siguen el mismo patrón: resta, valor absoluto, `cp HIT_SIZE`.
- Como el array está alineado, para pasar de `Y` a `X` bastaría `inc e` / `inc l`: el byte alto no cambia. Es una optimización opcional; `inc de` / `inc hl` también vale.

⚠ Errores típicos

Nos pasa a todos:

- Olvidar el valor absoluto: una resta negativa es un número grande (255) y nunca se solapa.
- Devolver el resultado en `Z`: este contrato usa el carry.

Ejercicio 10: sys_collision_one: ¿choca esta entidad con la sonda?

sys_collision_one

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks ONE entity (the candidate) against the probe, which is in
;; collision_probe. If the candidate has the COLLIDER component, its
;; KIND is one of the kinds in collision_mask, and it overlaps the probe,
;; then it is freed and collision_hits is incremented.
;; 📥 INPUT: DE: candidate entity
;; ❌ May modify A, DE, HL
sys_collision_one::
    ld   a, [de]                ;; \ A = candidate.CMP
    and  CMP_COLLIDER           ;; | Does it have COLLIDER?
    ret  z                      ;; / No: not for this system

    ld   hl, E_KIND             ;; \ HL = &candidate.KIND
    add  hl, de                 ;; |
    ld   a, [collision_mask]    ;; | A = collision_mask & candidate.KIND:
    and  [hl]                   ;; | is it a kind the probe collides with?
    ret  z                      ;; / No: ignore it

    ld   a, [collision_probe]   ;; \ HL = *(collision_probe)
    ld   l, a                   ;; |
    ld   a, [collision_probe + 1] ;; |
    ld   h, a                   ;; / = the probe

    push de                     ;; \ Save the candidate
    call sys_collision_overlaps        ;; | Do they overlap? (DE = candidate, HL =
probe)
    pop  de                     ;; / Restore it

    ret  nc                     ;; They do not overlap: nothing to do

    ld   hl, collision_hits     ;; \ One more collision:
    inc  [hl]                   ;; / collision_hits++

    jp   man_entity_free        ;; The candidate is removed (DE)
```

ⓘ Fíjate en

- Los filtros van de más barato a más caro: componente, tipo y, al final, el solape.
- `push de` / `pop de` protege la candidata: `sys_collision_overlaps` modifica `DE`.

⚠ Errores típicos

Nos pasa a todos:

- Comparar con `KIND == tipo` en vez de usar `AND` con la máscara: la sonda no podría chocar con dos tipos.
- Liberar la candidata antes de contar la colisión: `DE` ya no vale.

Ejercicio 11: scene_menu_update / scene_gameover_update: cambiar de escena con START

scene_menu_update

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Menu scene, every frame: if START has just been pressed, go to
;; the game scene (scene_game). If not, do nothing.
;; ⚠️ Use input_pressed (the "edge") and scene_change
scene_menu_update::
    ld   a, [input_pressed]     ;; \ A = the buttons just pressed
    and  BTN_START              ;; | Is START one of them?
    ret  z                      ;; / No: nothing to do

    ld   hl, scene_game         ;; \ HL = the game scene
    jp   scene_change           ;; / Change to it (and return)
```

scene_gameover_update

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Game over scene, every frame: if START has just been pressed, go to
;; the menu scene (scene_menu). If not, do nothing.
;; Each scene only updates its own things: this one does not touch the game.
scene_gameover_update::
    ld   a, [input_pressed]     ;; \ A = the buttons just pressed
    and  BTN_START              ;; | Is START one of them?
    ret  z                      ;; / No: nothing to do

    ld   hl, scene_menu         ;; \ HL = the menu scene
    jp   scene_change           ;; / Change to it (and return)
```

ⓘ Fíjate en

- Las dos rutinas son iguales salvo la escena de destino.
- `jp scene_change`, no `call`: después no queda nada por hacer en este fotograma.

⚠ Errores típicos

Nos pasa a todos:

- Usar `input_held`: START dura varios fotogramas y saltarías de escena más de una vez.

Ejercicio 12: scene_game_update / game_handle_collisions: el orden del juego

scene_game_update

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Game scene, every frame. First, the collisions; then, everything moves.
;; This way the collisions are checked with the positions that have just
;; been DRAWN, so the player sees the shot touch the asteroid (or the
;; asteroid touch the ship) before anything happens.
;;
;;  0. If the ship is gone (is_ship_gone: Z), it has finished exploding:
;;     go to the game over scene (scene_gameover).
;;     (This must be the first thing: nothing can have taken that slot yet.)
;;  1. game_handle_collisions: all the collisions of the game and what they cause.
;;  2. sys_input_update (sets VX from the D-pad), game_handle_shooting (A shoots),
;;     sys_spawn_update, sys_physics_update (it moves EVERYTHING: the ship, the
;;     asteroids and the shot), sys_shot_update (the physics may have freed the
;;     shot: it forgets it) and sys_selfdestroy_update.
;;  3. sys_prerender_update: prepares the sprites of how everything has ended
;;     up. The main loop copies them to the OAM in VBLANK.
scene_game_update::
    call is_ship_gone            ;; Z if the ship has gone
    jp   z, .game_over
    call game_handle_collisions  ;; 1. Collisions first, with what has been drawn
    call sys_input_update        ;; 2. Everything moves
    call game_handle_shooting
    call sys_spawn_update
    call sys_physics_update
    call sys_shot_update         ;; AFTER the physics: it may have freed the shot
    call sys_selfdestroy_update
    jp   sys_prerender_update    ;; 3. Prepare the sprites
.game_over
    ld   hl, scene_gameover
    jp   scene_change
```

game_handle_collisions

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; All the collisions of the game, and what they cause. The system
;; (sys_collision_update) only checks ONE probe against all the entities:
;; the rules of the game say who is the probe, who it collides with and what happens.
;;  1. The shot (if there is one, sys_shot_get) against the asteroids (KIND_ENEMY).
;;     If it hits any, call game_enemy_hit.
;;  2. The ship (SHIP_ENTITY) against the asteroids. If it hits any:
;;     call game_ship_destroyed. The ship becomes an explosion and the
;;     game goes on until the explosion disappears.
;;  3. The ship against the bonus items (KIND_BONUS).
;;     If it catches any, call game_bonus_taken.
;; ⚠️ sys_collision_update returns Z (none) / NZ (at least one)
;; ❌ May modify A, BC, DE, HL
game_handle_collisions::
    call sys_shot_get           ;; \ HL = the shot (Z if there is none)
    jr   z, .ship               ;; | No shot: skip this pass
    ld   b, KIND_ENEMY          ;; | B = it collides with asteroids
    call sys_collision_update   ;; | A = how many it has hit
    call nz, game_enemy_hit     ;; / Some: points, explosion, bonus...

.ship
    ld   hl, SHIP_ENTITY        ;; \ HL = the ship (the probe)
    ld   b, KIND_ENEMY          ;; | B = it collides with asteroids
    call sys_collision_update   ;; | A = how many it has hit
    call nz, game_ship_destroyed ;; / Some: the ship becomes an explosion

    ld   hl, SHIP_ENTITY        ;; \ HL = the ship (the probe)
    ld   b, KIND_BONUS          ;; | B = it collides with bonus items
    call sys_collision_update   ;; | A = how many it has caught
    call nz, game_bonus_taken   ;; / Some: points

    ret
```

ⓘ Fíjate en

- `is_ship_gone` va primero: sin nave, no tiene sentido hacer nada más.
- Las colisiones van antes de mover, con lo que ya se ha dibujado.
- Cada pasada deja en `A` el número de colisiones y el flag `Z`: `call nz, regla` solo la llama si hubo.

⚠ Errores típicos

Nos pasa a todos:

- Poner las colisiones después de mover: el jugador no ve el contacto antes de que pase algo.
- Llamar a `game_enemy_hit` sin disparo: `sys_shot_get` da `Z` cuando no lo hay.